

# Guia Flutter — Do Zero ao App de Flashcards

---

Material criado para quem já conhece React/TypeScript mas nunca tocou em Flutter.  
Cada conceito novo é explicado comparando com o equivalente React que você já conhece.

## Índice

---

1. O que é Flutter e como funciona
  2. Como rodar o projeto
  3. Estrutura de pastas
  4. Dart: a linguagem do Flutter
  5. Widgets: a base de tudo
  6. Estado: StatefulWidget e setState
  7. Ciclo de vida: initState e dispose
  8. Navegação com go\_router
  9. Chamadas HTTP e a API
  10. Armazenamento local: SharedPreferences
  11. Layout: Column, Row e Container
  12. Animações
  13. Models: Deck e Flashcard
  14. Tabela de equivalências React → Flutter
- 

## 1. O que é Flutter e como funciona

---

Flutter é um framework do Google para criar apps mobile (Android/iOS), desktop (Windows/Mac/Linux) e web a partir de **um único código** escrito em **Dart**.

A diferença fundamental para o React:

| React                        | Flutter                                          |
|------------------------------|--------------------------------------------------|
| Roda no navegador (DOM)      | Renderiza tudo com o motor gráfico Skia/Impeller |
| Componentes → elementos HTML | Widgets → pixels desenhados diretamente          |
| JavaScript / TypeScript      | Dart                                             |
| CSS para estilo              | Propriedades inline nos próprios Widgets         |

No React, um `<div>` vira um elemento HTML real. No Flutter, cada Widget é desenhado diretamente na tela — não tem HTML nem CSS por baixo.

## 2. Como rodar o projeto

### Pré-requisitos

- Flutter instalado em `C:\develop\flutter\`
- Backend FastAPI rodando ( `uvicorn main:app --reload` na pasta `backend/` )

### Adicionar Flutter ao PATH (faça isso uma vez)

1. Pesquise "**variáveis de ambiente**" no Windows
2. Em **Variáveis do sistema** → `Path` → **Editar** → **Novo**
3. Adicione: `C:\develop\flutter\bin`
4. Abra um **novo** terminal e teste: `flutter --version`

Depois disso você pode usar `flutter` diretamente sem o caminho completo.

### Rodando no Windows Desktop (mais rápido para testar)

```
# 1. Abra um terminal dentro da pasta flutter_frontend
cd flutter_frontend

# 2. Rode no Windows
flutter run -d windows
```

### Rodando no Chrome/Edge (alternativa)

```
flutter run -d edge
```

## Rodando no Android (depois de instalar o Android Studio)

```
# 1. Abra o Android Studio → Device Manager → crie um emulador
# 2. Inicie o emulador
# 3. Execute:
flutter run -d android
```

### Por que 10.0.2.2 no Android?

O emulador Android roda em uma máquina virtual isolada. Para ele, 127.0.0.1 é a própria VM, não o seu PC. O endereço 10.0.2.2 é o atalho que o emulador usa para acessar o localhost do PC onde ele está rodando.

O código já detecta a plataforma automaticamente — você não precisa mudar nada.

## Hot Reload e Hot Restart

Uma das melhores features do Flutter: você edita o código e vê a mudança **instantaneamente** sem perder o estado do app.

- **Hot Reload** ( `r` no terminal): atualiza apenas o que mudou, mantém o estado
- **Hot Restart** ( `R` no terminal): reinicia o app do zero
- **Quit** ( `q` no terminal): encerra

## 3. Estrutura de pastas

```
flutter_frontend/
├── lib/                                ← TODO o código Dart fica aqui
│   ├── main.dart                      ← ponto de entrada + roteador
│   ├── models/
│   │   ├── deck.dart                 ← classe de dados Deck
│   │   └── flashcard.dart            ← classe de dados Flashcard
│   ├── services/
│   │   └── api_service.dart          ← todas as chamadas HTTP
│   └── screens/
│       ├── splash_screen.dart         ← tela inicial com loading
│       ├── welcome_screen.dart        ← tela de login (input de nome)
│       └── home_screen.dart           ← lista de decks
```

```

├── study_screen.dart  ← flashcard com flip
├── android/           ← configurações do build Android
├── windows/           ← configurações do build Windows
├── web/               ← configurações do build Web
├── pubspec.yaml       ← equivalente ao package.json
└── FLUTTER_GUIA.md    ← este arquivo

```

Você vai trabalhar **quase exclusivamente dentro de `lib/`**. As outras pastas são geradas automaticamente e raramente precisam ser tocadas.

## 4. Dart: a linguagem do Flutter

Dart é parecido com TypeScript. Se você já sabe TS, vai pegar rápido.

### Variáveis

```

// Dart
String nome = 'Alexandre';
int idade = 25;
bool ativo = true;
double preco = 9.99;

// Tipo inferido (igual ao let/const do JS)
var nome = 'Alexandre';
final nome = 'Alexandre'; // imutável após atribuição (como const)
const PI = 3.14;           // constante de compilação

```

### Null safety

Dart tem null safety nativo — você precisa declarar explicitamente se uma variável pode ser nula:

```

String nome = 'Alexandre'; // NUNCA pode ser null
String? nome = null;       // PODE ser null (o ? indica isso)

// Para usar uma variável nullable, você precisa verificar:
String? userName = prefs.getString('userName');
print(userName ?? 'Padrão'); // ?? = valor padrão se null (igual ao || no JS)
print(userName!);           // ! = "tenho certeza que não é null" (perigoso)

```

### Funções async/await

Idêntico ao JavaScript:

```
// JavaScript
async function buscarDecks() {
  const res = await fetch('/decks');
  const data = await res.json();
  return data;
}

// Dart
Future<List<Deck>> buscarDecks() async {
  final res = await http.get(Uri.parse('http://127.0.0.1:8000/decks'));
  final data = jsonDecode(res.body) as List;
  return data.map((d) => Deck.fromJson(d)).toList();
}
```

`Future<T>` em Dart = `Promise<T>` em TypeScript.

## Listas e Maps

```
// Lista (equivale ao Array do JS)
List<String> nomes = ['Alice', 'Bob'];
nomes.add('Carlos');
nomes.removeWhere((n) => n == 'Bob');
nomes.map((n) => n.toUpperCase()).toList();

// Map (equivale ao Object/Record do TS)
Map<String, dynamic> deck = {'id': 1, 'nome': 'Hiragana'};
deck['nome']; // acesso por chave
```

## Classes

```
class Deck {
  final int id;
  final String nome;

  // Construtor com parâmetros nomeados
  Deck({required this.id, required this.nome});
}

// Uso:
final deck = Deck(id: 1, nome: 'Hiragana');
```

## 5. Widgets: a base de tudo

Em React, tudo é **componente**. Em Flutter, tudo é **Widget**.

```
// React
function BotaoVermelho({ texto, onClick }) {
  return <button onClick={onClick} style={{ color: 'red' }}>{texto}</button>;
}
```

```
// Flutter
class BotaoVermelho extends StatelessWidget {
  final String texto;
  final VoidCallback onTap; // VoidCallback = função que não retorna nada

  const BotaoVermelho({required this.texto, required this.onTap});

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: onTap,
      child: Text(texto, style: const TextStyle(color: Colors.red)),
    );
  }
}
```

### StatelessWidget vs StatefulWidget

| Tipo                         | Quando usar                    | Equivalente React                              |
|------------------------------|--------------------------------|------------------------------------------------|
| <code>StatelessWidget</code> | Componente sem estado interno  | Componente funcional sem <code>useState</code> |
| <code>StatefulWidget</code>  | Componente com estado que muda | Componente com <code>useState</code>           |

```
// StatelessWidget: recebe dados, exibe, não muda
class NomeDoUser extends StatelessWidget {
  final String nome;
  const NomeDoUser({required this.nome});

  @override
  Widget build(BuildContext context) => Text(nome);
}

// StatefulWidget: tem estado próprio que pode mudar
class Contador extends StatefulWidget {
```

```

@override
State<Contador> createState() => _ContadorState();
}

class _ContadorState extends State<Contador> {
  int _count = 0; // estado interno

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: () => setState(() => _count++),
      child: Text('$_count'),
    );
  }
}

```

## 6. Estado: StatefulWidget e setState

### No React

```

const [decks, setDecks] = useState<Deck[]>([]);

function adicionarDeck(novo: Deck) {
  setDecks(prev => [...prev, novo]);
}

```

### No Flutter

```

// Variável de estado declarada na classe _State
List<Deck> _decks = [];

// Para alterar o estado, envolva em setState()
void _adicionarDeck(Deck novo) {
  setState(() {
    _decks.add(novo);
  });
}

```

**A regra de ouro:** sempre que quiser atualizar a tela, coloque a mudança dentro de `setState(() { ... })`. Isso é equivalente a chamar `setDecks( ... )` no React — sem ele, a tela não re-renderiza.

## 7. Ciclo de vida: initState e dispose

### No React

```
useEffect(() => {  
  buscarDecks(); // roda ao montar o componente  
}, []);          // [] = só uma vez  
  
useEffect(() => {  
  return () => {  
    controller.dispose(); // roda ao desmontar (cleanup)  
  };  
}, []);
```

### No Flutter

```
@override  
void initState() {  
  super.initState();  
  _buscarDecks(); // equivale ao useEffect(fn, [])  
}  
  
@override  
void dispose() {  
  _controller.dispose(); // equivale ao return () => cleanup no useEffect  
  super.dispose();  
}
```

- `initState()` roda **uma vez** quando o widget entra na tela
- `dispose()` roda quando o widget é **removido** da tela
- Sempre chamar `super.initState()` e `super.dispose()` — é obrigatório

## 8. Navegação com go\_router

### No React



```
// Configuração
const router = createBrowserRouter([
  { path: '/', Component: Splash },
  { path: '/home', element: <Home /> },
  { path: '/deck/:deckId', element: <Study /> },
]);

// Uso nos componentes
const navigate = useNavigate();
navigate('/home');
navigate(`/deck/${deck.id}`);

// Parâmetros de rota
const { deckId } = useParams();
```

## No Flutter com go\_router

```
// Configuração no main.dart
final _router = GoRouter(
  routes: [
    GoRoute(path: '/', builder: (context, _) => const SplashScreen()),
    GoRoute(path: '/home', builder: (context, _) => const HomeScreen()),
    GoRoute(
      path: '/study/:deckId',
      builder: (context, state) => StudyScreen(
        deckId: int.parse(state.pathParameters['deckId']!),
      ),
    ),
  ],
);

// Uso nos Widgets
context.go('/home'); // equivale ao navigate('/home')
context.go('/study/${deck.id}'); // com parâmetro

// Parâmetros: passados direto no construtor do Widget (não via hook)
// StudyScreen recebe: final int deckId;
```

## 9. Chamadas HTTP e a API

### No React

```
// GET
const res = await fetch('http://127.0.0.1:8000/decks');
const data = await res.json();

// POST
const res = await fetch('http://127.0.0.1:8000/decks', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ nome: 'Hiragana', user_id: 1 }),
});
```

## No Flutter

```
import 'dart:convert';
import 'package:http/http.dart' as http;

// GET
final res = await http.get(Uri.parse('http://127.0.0.1:8000/decks'));
final data = jsonDecode(res.body) as List;

// POST
final res = await http.post(
  Uri.parse('http://127.0.0.1:8000/decks'),
  headers: {'Content-Type': 'application/json'},
  body: jsonEncode({'nome': 'Hiragana', 'user_id': 1}),
);

// DELETE
await http.delete(Uri.parse('http://127.0.0.1:8000/decks/1'));
```

No projeto, todas essas chamadas estão centralizadas em `lib/services/api_service.dart` para não ficar espalhado nas telas (boa prática equivalente a criar um arquivo `api.ts` no React).

## 10. Armazenamento local: SharedPreferences

### No React

```
// Salvar
localStorage.setItem('userName', 'Alexandre');
localStorage.setItem('isAuthenticated', 'true');
```

```
// Ler
const nome = localStorage.getItem('userName') ?? 'Estudante';
const logado = localStorage.getItem('isAuthenticated') === 'true';

// Remover
localStorage.removeItem('userName');
```

## No Flutter

```
final prefs = await SharedPreferences.getInstance();

// Salvar (tipos separados: setString, setBool, setInt)
await prefs.setString('userName', 'Alexandre');
await prefs.setBool('isAuthenticated', true);

// Ler
final nome = prefs.getString('userName') ?? 'Estudante';
final logado = prefs.getBool('isAuthenticated') ?? false;

// Remover
await prefs.remove('userName');
```

Diferença importante: em Flutter os métodos são **assíncronos** — você precisa de `await`. No `localStorage` do React, tudo é síncrono.

## 11. Layout: Column, Row e Container

No React você usa `div` + Tailwind/CSS. No Flutter, o layout é feito com Widgets específicos:

| CSS / Tailwind                                  | Flutter                                                               |
|-------------------------------------------------|-----------------------------------------------------------------------|
| <code>flex-col</code>                           | <code>Column</code>                                                   |
| <code>flex-row</code>                           | <code>Row</code>                                                      |
| <code>&lt;div&gt;</code> com padding/margin/cor | <code>Container</code> ou <code>Padding</code>                        |
| <code>gap-4</code> entre itens                  | <code>SizedBox(height: 16)</code> ou <code>SizedBox(width: 16)</code> |
| <code>grid-cols-2</code>                        | <code>GridView.builder</code> com <code>crossAxisCount: 2</code>      |
| <code>overflow-y-auto</code>                    | <code>ListView</code> ou <code>SingleChildScrollView</code>           |

| CSS / Tailwind                                       | Flutter                                                              |
|------------------------------------------------------|----------------------------------------------------------------------|
| <code>bg-gradient-to-br</code>                       | <code>BoxDecoration</code> com <code>LinearGradient</code>           |
| <code>rounded-xl</code>                              | <code>BorderRadius.circular(12)</code>                               |
| <code>border border-white/20</code>                  | <code>Border.all(color: Colors.white.withValues(alpha: 0.2))</code>  |
| <code>text-white/60</code>                           | <code>Colors.white.withValues(alpha: 0.6)</code>                     |
| <code>w-full</code> num filho de <code>Column</code> | <code>width: double.infinity</code>                                  |
| <code>flex-1</code> (ocupa espaço restante)          | <code>Expanded(child: ...)</code>                                    |
| <code>items-center justify-center</code>             | <code>mainAxisAlignment: .center, crossAxisAlignment: .center</code> |

## Exemplo prático

```
// React com Tailwind
<div className="flex flex-col items-center gap-4 p-6 bg-red-600 rounded-xl">
  <h1 className="text-white text-2xl">Olá!</h1>
  <button className="bg-white text-red-600 p-3 rounded-lg w-full">Entrar</button>
</div>
```

```
// Flutter equivalente
Container(
  padding: const EdgeInsets.all(24),
  decoration: BoxDecoration(
    color: const Color(0xFFDC2626),
    borderRadius: BorderRadius.circular(12),
  ),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.center,
    children: [
      const Text('Olá!',
        style: TextStyle(color: Colors.white, fontSize: 24)),
      const SizedBox(height: 16),
      SizedBox(
        width: double.infinity,
        child: ElevatedButton(
          onPressed: () {},
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.white,
```

```

        foregroundColor: const Color(0xFFDC2626),
      ),
      child: const Text('Entrar'),
    ),
  ],
),
)

```

## 12. Animações

### No React com framer-motion

```

<motion.div
  initial={{ opacity: 0, y: 20 }}
  animate={{ opacity: 1, y: 0 }}
  whileTap={{ scale: 0.95 }}
/>

```

### No Flutter

Para animações simples, Flutter tem widgets prontos:

```

// Fade + slide de entrada
AnimatedOpacity(
  opacity: _visible ? 1.0 : 0.0,
  duration: const Duration(milliseconds: 300),
  child: Text('Olá'),
)

// Escala ao pressionar (equivale ao whileTap)
GestureDetector(
  onTapDown: (_) => setState(() => _pressed = true),
  onTapUp: (_) => setState(() => _pressed = false),
  child: AnimatedScale(
    scale: _pressed ? 0.95 : 1.0,
    duration: const Duration(milliseconds: 100),
    child: ElevatedButton(...),
  ),
)

```

## Animação de flip 3D (usada no flashcard)

Para animações complexas como o flip do card, usamos `AnimationController` + `Transform`:

```
// 1. Declarar o controller (no initState)
late AnimationController _flipController;
late Animation<double> _flipAnimation;

_flipController = AnimationController(
  duration: const Duration(milliseconds: 400),
  vsync: this, // 'this' = o próprio widget controla o tick de animação
);

_flipAnimation = Tween<double>(begin: 0, end: 1).animate(
  CurvedAnimation(parent: _flipController, curve: Curves.easeInOut),
);

// 2. Disparar a animação
_flipController.forward(); // vai de 0 → 1
_flipController.reverse(); // vai de 1 → 0

// 3. Usar no build com AnimatedBuilder
AnimatedBuilder(
  animation: _flipAnimation,
  builder: (context, child) {
    final angle = _flipAnimation.value * pi; // 0 a 180 graus
    return Transform(
      transform: Matrix4.identity()
        ..setEntry(3, 2, 0.001) // perspectiva 3D
        ..rotateY(angle), // rotação no eixo Y
      alignment: Alignment.center,
      child: Container(...),
    );
  },
)
```

## 13. Models: Deck e Flashcard

Models são classes que definem o formato dos dados que vêm da API. No React você usava `any[]` sem tipo definido; em Dart a gente tipifica explicitamente.

**deck.dart**

```

class Deck {
  final int id;
  final String nome;
  final int userId;
  final List<Flashcard> cards;

  Deck({required this.id, required this.nome, required this.userId, this.cards = const List<Flashcard>()});

  // fromJson: converte o Map (JSON decodificado) em um objeto Deck
  factory Deck.fromJson(Map<String, dynamic> json) {
    return Deck(
      id: json['id'],
      nome: json['nome'],
      userId: json['user_id'] ?? 1,
      cards: json['cards'] != null
        ? (json['cards'] as List).map((c) => Flashcard.fromJson(c)).toList()
        : [],
    );
  }
}

```

A palavra **factory** indica um construtor especial que pode retornar instâncias de subtipos ou fazer lógica de criação — aqui usamos para criar um **Deck** a partir de um **Map<String, dynamic>** (o JSON).

## 14. Tabela de equivalências React → Flutter

| React / JS                                                     | Flutter / Dart                                                   |
|----------------------------------------------------------------|------------------------------------------------------------------|
| <code>useState(valor)</code>                                   | variável na classe <code>_State</code> + <code>setState()</code> |
| <code>useEffect(fn, [])</code>                                 | <code>initState()</code>                                         |
| <code>useEffect(() =&gt; cleanup, [])</code>                   | <code>dispose()</code>                                           |
| <code>useNavigate()</code> +<br><code>navigate('/rota')</code> | <code>context.go('/rota')</code>                                 |
| <code>useParams()</code>                                       | parâmetro no construtor do Widget                                |
| <code>localStorage.setItem</code>                              | <code>prefs.setString</code> / <code>setBool</code>              |
| <code>localStorage.getItem</code>                              | <code>prefs.getString</code> / <code>getBool</code>              |

| React / JS                               | Flutter / Dart                                                                               |
|------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>fetch(url)</code>                  | <code>http.get(Uri.parse(url))</code>                                                        |
| <code>JSON.stringify(obj)</code>         | <code>jsonEncode(map)</code>                                                                 |
| <code>res.json()</code>                  | <code>jsonDecode(res.body)</code>                                                            |
| <code>Promise&lt;T&gt;</code>            | <code>Future&lt;T&gt;</code>                                                                 |
| <code>async/await</code>                 | <code>async/await</code> (idêntico)                                                          |
| <code>null ?? 'padrão'</code>            | <code>null ?? 'padrão'</code> (idêntico)                                                     |
| <code>array.map(fn)</code>               | <code>list.map(fn).toList()</code>                                                           |
| <code>array.filter(fn)</code>            | <code>list.where(fn).toList()</code>                                                         |
| <code>[ ... arr, item]</code>            | <code>[ ... list, item]</code> (idêntico)                                                    |
| <code>&lt;div&gt;</code> com flexbox     | <code>Column</code> / <code>Row</code>                                                       |
| <code>&lt;div&gt;</code> com padding/cor | <code>Container</code>                                                                       |
| <code>gap-4</code>                       | <code>SizeBox(height/width: 16)</code>                                                       |
| <code>flex-1</code>                      | <code>Expanded</code>                                                                        |
| <code>w-full</code>                      | <code>width: double.infinity</code>                                                          |
| <code>&lt;input&gt;</code>               | <code>TextField</code> com <code>TextEditingController</code>                                |
| <code>&lt;button&gt;</code>              | <code>ElevatedButton</code> / <code>TextButton</code> / <code>OutlinedButton</code>          |
| <code>onClick</code>                     | <code>onPressed</code> / <code>onTap</code>                                                  |
| <code>className=" ... "</code>           | propriedades no Widget (sem CSS)                                                             |
| <code>key={item.id}</code>               | não necessário em <code>ListView</code> / <code>GridView</code> com <code>itemBuilder</code> |
| <code>import './Component'</code>        | <code>import '../screens/home_screen.dart'</code>                                            |

Documento gerado durante a migração do frontend React para Flutter — Maio 2026